

事前課題：ヴァイオリン・同期

氏名：仲里偉武輝
学籍番号：25G1090

2026 年 5 月 13 日

第 1 章

基本設計：ヴァイオリン

1.1 プロジェクト概要

本プロジェクトは，Arduino を用いたサーバー・クライアント方式により，複数の楽器が低遅延で同期し，輪唱演奏を実現するシステムの開発を目的としている．5 台の Arduino を接続して，聴覚上で遅延を認知できないレベルを目指すものである．本プログラムで設計を行うヴァイオリン・クライアントは，この合奏システムの一翼を担う重要なコンポーネントである．本クライアントは，サーバーから無線送信される拍情報に基づき，Arduino でリアルタイムに演奏データを生成し，PC 上の Processing でヴァイオリン特有の音色を合成して再生する役割を果たす．

1.2 機能一覧

本プロジェクトにおいて，ヴァイオリン・クライアントが担う主要な機能と，機能分解構造 (FBS) との対応を表 1.1 に示す．

表 1.1 FBS と機能一覧の対応 (ヴァイオリン)

FBS 番号	機能名	役割
F1	同期信号受信機能	サーバー (親機) から無線通信を介して送出される同期信号 (拍番号, BPM) をリアルタイムに受信する．
F2	譜面参照・音高決定機能	受信した拍番号を元に，内部の譜面配列データと照合し，その瞬間に発音すべき音の高さを決定する．
F3	テンポ計算機能	受信した BPM 情報を基に，一拍の物理的な長さをミリ秒単位で動的に計算し，テンポを同期させる．
F4	Processing 送信機能	決定された音の高さと長さのデータを，シリアル通信を用いて PC 上の Processing へ即座に送信する．
F5	音色合成・再生機能	Processing 上で受信したデータをもとに，ヴァイオリン特有の擦弦音を数学的に合成し，スピーカから再生する．
F6	例外処理機能	シリアル通信のノイズやデータ欠損を検知し，不正な形式のデータによるシステム停止を防ぐ．

1.3 システム構成図

本システムはサーバー Arduino を親機とし，各楽器 Arduino が子機として無線接続されるスター型ネットワーク構成をとる．ヴァイオリン担当の構成範囲は以下の通りである．

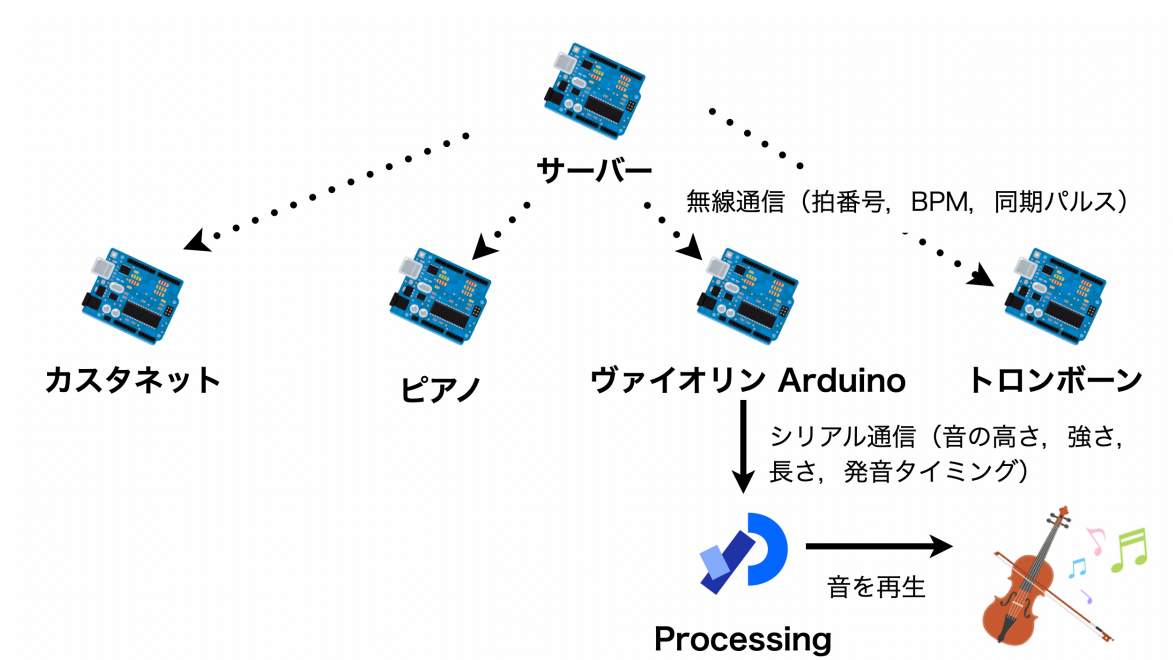


図 1.1 システム構成図

1.4 操作インターフェース設計

本クライアントシステムは、合奏の同期を維持するため、サーバーの制御下で全自動動作を行う設計としている。

1.4.1 操作方法

ユーザーは Arduino の電源を投入する以外の操作を必要としない。演奏の開始、停止、テンポ変更はすべてサーバーからの無線信号によって制御される。

1.5 状態遷移図

ヴァイオリン・クライアントの内部状態の遷移を図 1.2 に示す。

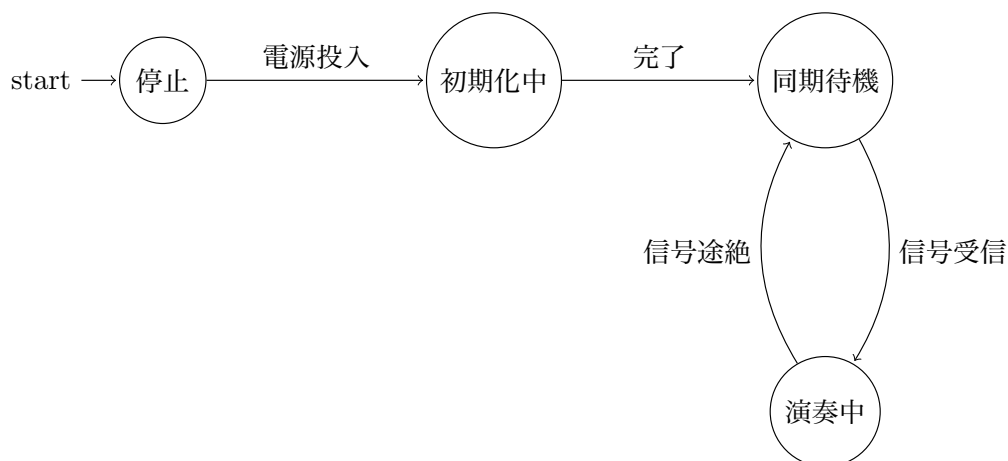


図 1.2 で定義した各状態の役割と、遷移のトリガーとなる条件を以下に詳しく定義する。

停止 (Stopped) 電源が投入されていない、またはシステムが完全に停止している初期状態である [cite: 832]。ユーザーが Arduino に電源を投入することで「初期化中」へ遷移する。

初期化中 (Initializing) 電源投入直後、シリアル通信のセットアップや無線通信ライブラリの開始処理を行っている状態である。すべての初期設定が正常に完了すると、自動的に「同期待機」へ遷移する。

同期待機 (Sync Waiting) サーバーからの同期パケット（拍番号, BPM）を待機している状態である。この間、演奏は行われない。サーバーからの有効な信号を正常に受信すると「演奏中」へ遷移し、合奏に加わる。

演奏中 (Playing) 受信した同期信号に基づき、リアルタイムで演奏データを生成し Processing へ送信しているメインの状態である。演奏中に一定時間サーバーからのパケットが途絶えた（信号途絶）場合は、例外処理として自動的に「同期待機」へ戻り、再びサーバーとの同期

確立を試みる.

第 2 章

詳細設計：ヴァイオリン

2.1 部品一覧

本システムを構成する物理的な部品は、メイン制御ユニットである Arduino UNO R4 WiFi、音響処理および出力ユニットである MacBook Air、そして通信および給電に用いる USB Type-C ケーブルの 3 点で構成される。

2.2 ハードウェア設計

本システムは外部回路を使用せず、Arduino UNO R4 WiFi と MacBook Air を USB ケーブルで直結する最小構成とする。電源は USB 経由で MacBook Air から Arduino へ供給し、同時にシリアル通信のデータ転送経路として利用する。

2.3 ソフトウェア設計とファイル構成

2.3.1 ソフトウェア構成とデータフロー

本システムのソフトウェア構成および、各モジュール間でのデータの受け渡しを図 2.1 に示す。

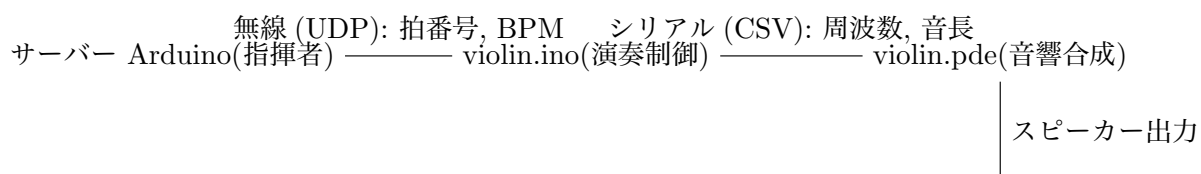


図 2.1 ソフトウェア構成とデータフロー図

2.3.2 モジュール別の役割詳細

図 2.1 に基づき、各ソフトウェアの責務を以下のように定義する。

- **violin.ino (Arduino 側):** 無線パケットを監視し、受信した「拍番号」を内部の楽譜配列のインデックスとして変換する。また「BPM」からミリ秒単位の音長を算出し、Processing が解釈可能な CSV 形式に整形する役割を持つ。
- **violin.pde (Processing 側):** シリアルポートを常時監視し、受信した文字列を解析 (パース) して音響パラメータを抽出する。Minim ライブラリを用いてヴァイオリンの物理モデルに基づいた波形をリアルタイムに生成・出力する役割を持つ。

2.4 内部データ構造の定義

本システムでは、演奏データを効率的に処理するために以下のデータ構造を使用する。Arduino 側では、音高を示す周波数情報を格納した整数型の楽譜配列 (melody 配列) を保持する。この配列のインデックスはサーバーから受信した拍番号に対応する。Processing 側では、受信したシリアル文字列を分割し、一時的に浮動小数点数型の配列に格納することで、音高や音長として個別に処理可能な形式に変換する。

2.5 関数設計と内部仕様

各プログラムで実装する主要な関数の詳細な仕様を以下に定義する。

2.5.1 violin.ino (Arduino 側)

setup 関数はプログラム開始時に一度のみ実行され、引数および戻り値は持たない。この関数内では、シリアル通信の速度設定や無線通信ライブラリの初期化処理を行い、サーバーからのパケット受信を待機可能な状態にする。

receiveSyncData 関数は、サーバーから送信される無線パケットを監視し、受信に成功した際に実行される。この関数は引数を持たず、受信成功の有無を真偽値で返す仕様とする。内部処理では、受信したデータパケットから「拍番号」「BPM」という 2 つの整数値を抽出し、それぞれ対応するグローバル変数へ格納する。これにより、他の関数から最新の同期情報を参照することが可能となる。

sendNoteToPC 関数は、同期信号を受信した直後に呼び出される関数であり、引数および戻り値は持たない。この関数は receiveSyncData 関数によって更新されたグローバル変数の拍番号を用いて、楽譜配列から該当する音の周波数を読み出す。同時に、BPM の値から一拍の長さをミリ秒単位で算出し、音高と音長の 2 つの数値をカンマ区切りの文字列としてシリアルポートへ出力する。

2.5.2 violin.pde (Processing 側)

serialEvent 関数は、Arduino からシリアルポートに新しいデータが届いた際にシステムによって自動的に呼び出されるイベントハンドラである。引数として受信元となる Serial クラスのオブジェクトを受け取り、戻り値は持たない。この関数内では、改行コードを区切りとして一行の文字列を読み込み、trim 関数によって不要な空白を除去した後、split 関数を用いてカンマ区切りの数値を個別のデータとして分離する。その際、通信のノイズやデータの欠損によるシステム停止を防ぐ例外処理として、分割されたデータ配列の長さが正確に 2 つであるかを判定する。この条件を満たした場合のみ、playNoteData 関数へデータを渡し、演奏を実行する設計としている。

playNoteData 関数は、前述の判定を通過して分離された音高と音長を引数として受け取り、演奏を実行する。具体的には、第一引数に音高、第二引数に音長を指定する。内部では、あらかじめ定義された ViolinInstrument クラスのインスタンスを生成し、指定された周波数と ADSR エンベロープを用いて、ヴァイオリン特有の擦弦音を再生する処理を行う。

2.6 オブジェクト設計 (クラス図)

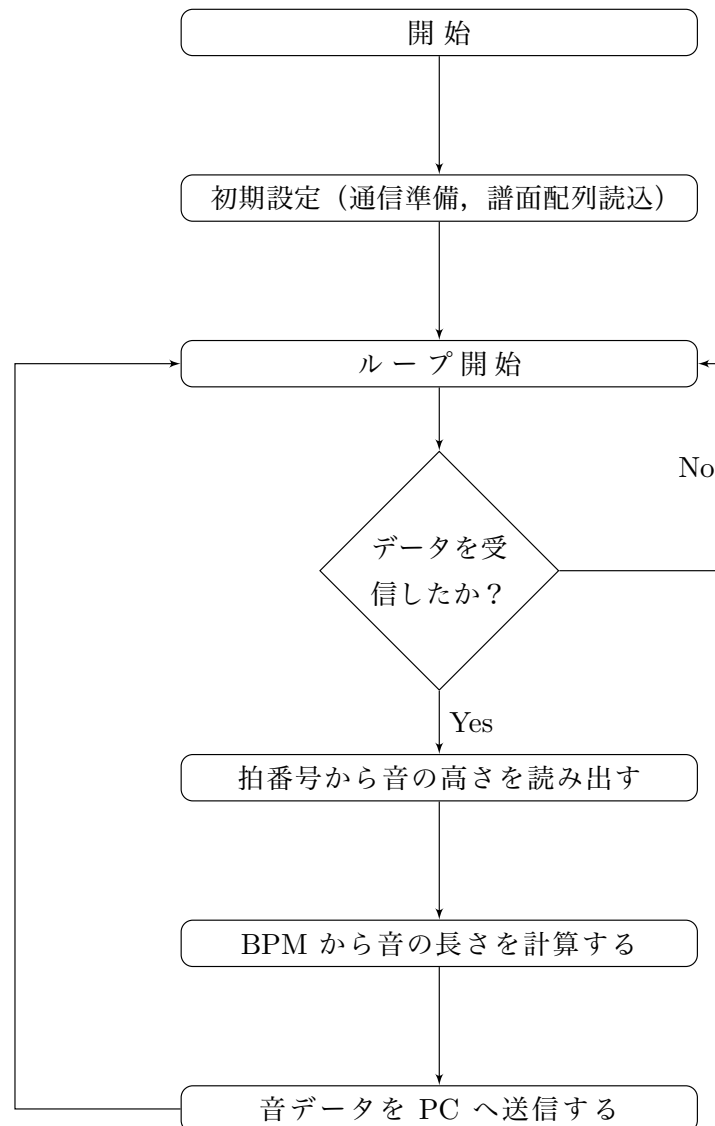
Processing 側で楽器の音色を生成するための ViolinInstrument クラスの構造を以下に示す.

ViolinInstrument
- wave: Oscil - ampEnv: ADSR + maxAmp: float
+ ViolinInstrument(frequency: float, amplitude: float) + noteOn(duration: float): void + noteOff(): void

このクラスは、オシレーターによる倍音成分の生成と、エンベロープによるヴァイオリン特有の擦弦楽器らしい音の立ち上がりと減衰を制御する役割を持つ.

2.7 処理フローチャート

Arduino 側である violin.ino のメイン処理の流れを以下に示す.



第 3 章

基本設計：同期

3.1 システムの概要

本システムは，Arduino オーケストラ全体のリズムを統括する「指揮者（サーバー）」として機能するリズム管理システムである．全体の演奏テンポ（BPM）および現在の小節番号を中央で一元管理し，Wi-Fi の UDP ローカルブロードキャスト通信を用いて，全クライアント（各楽器の Arduino）へ同期パルスを送信する役割を担う．通信遅延やパケットロスを許容し，再送処理を行わず音楽のテンポ維持を最優先する堅牢な設計を特徴とする．

3.2 機能一覧と FBS の対応

本同期システムが担う主要な機能と、機能分解構造（FBS）との対応を表 3.1 に示す。表 3.1 は、同期システムに必要な要件を機能単位で整理したものである。F1 は時間の管理とパケット送信による指揮機能を定義し、F2 および F3 はユーザーとの接点となる変更受付と視覚的提示を担う。また、F4 は通信の不確実性を前提とした実用的なエラーハンドリングの方針を示している。

表 3.1 FBS と機能一覧の対応（同期サーバー担当）

FBS 番号	機能名	役割
F1	テンポ管理・小節送信機能	現在の BPM に基づいて 1 小節の時間間隔を計算し、到達時刻ごとに小節番号を UDP ブロードキャストで全クライアントへ一斉送信する。
F2	BPM 変更受付機能	物理スイッチの押下を検知し、現在の BPM を「+5」または「-5」変動させ、新たな BPM 値を全クライアントへ通告する。
F3	BPM 視覚表示機能	現在設定されている BPM の数値を LED マトリクスにリアルタイムに描画し、ユーザーに視覚的に提示する。
F4	通信エラーハンドリング機能	UDP 通信の性質上発生しうるパケットロスに対して、あえて再送処理を行わず、音楽のリアルタイム性を優先してパケットを破棄する。

3.2.1 システム構成図

本システムの全体構成を図 3.1 に示す。図 3.1 は、リズム管理サーバーを中心に、入力デバイス、出力デバイス、および各楽器クライアントとの通信関係を可視化したものである。

中央に配置されたサーバー機（Arduino Uno R4 WiFi）は、システム全体の指揮者として機能する。サーバーには入力系として BPM 調整用の 2 つのタクトスイッチが接続され、ハードウェアによるチャタリング防止回路を介して BPM の増減指示を受け取る。また、出力系として内蔵の LED マトリクスを用いて、現在の BPM 値をユーザーへリアルタイムにフィードバックする。

ネットワーク面では、サーバーは Wi-Fi ルーターを介してローカルネットワークを構築し、UDP プロトコルを用いたブロードキャスト送信（ポート番号 3000）を行う。これにより、ヴァイオリンやピアノを含む全パートの楽器クライアントに対して、時間差のない正確な同期信号を一斉に配信する構造となっている。

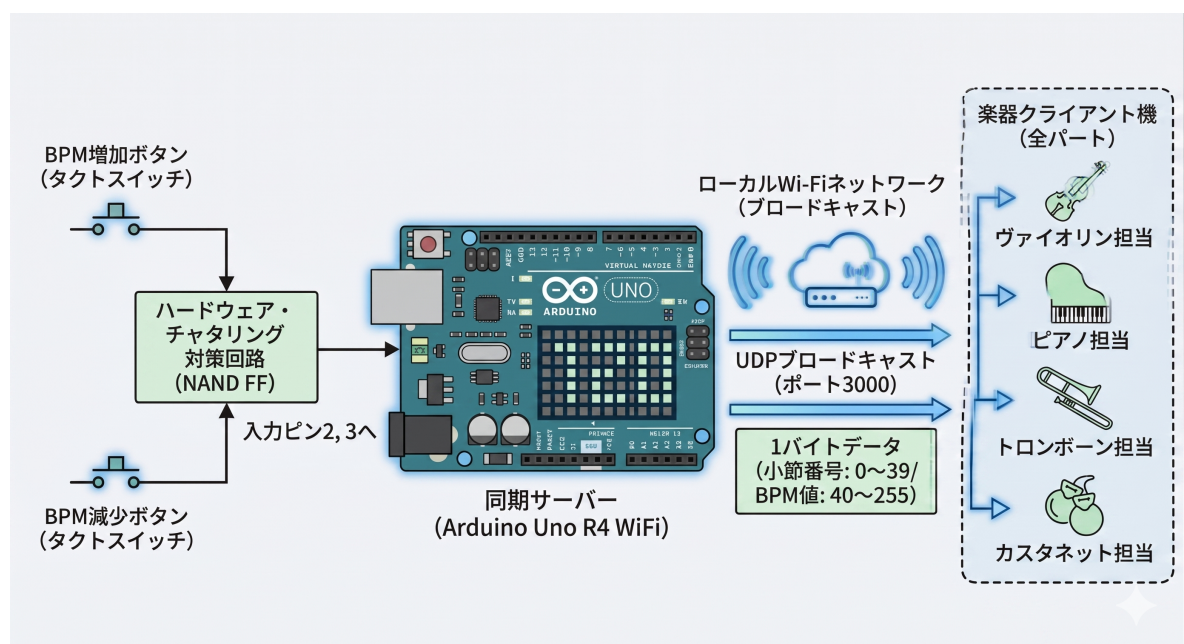


図 3.1 同期システムの全体構成図

3.3 操作インターフェース設計

本システムの操作は、BPM を調整するための 2 つの物理的なタクトスイッチによって行う。アップボタンは 1 回押下するごとに BPM を 5 増加させ、ダウンボタンは 5 減少させる挙動となる。ユーザーがスイッチを操作すると、Arduino Uno R4 WiFi の内蔵 LED マトリクスに現在の BPM 値が即座に表示され、視覚的なフィードバックを得ることができる。通信負荷を軽減するため、ボタンの連続押下時は最後の操作から 1 秒間が経過したタイミングでクライアントへの送信が行われる仕様としている。

3.4 状態遷移設計

本システムにおけるサーバー Arduino の内部状態の遷移を図 3.2 に示す。図 3.2 は、電源投入から各状態へ至る論理的な流れを可視化したものである。システムは初期化完了後、通常稼働状態において同期信号の送出ループを維持する。ユーザーによるスイッチ入力が発知された際のみ、BPM 変更処理状態へと遷移し、パラメータの再計算と表示更新を完了させた後に再び通常稼働へと復帰する設計としている。

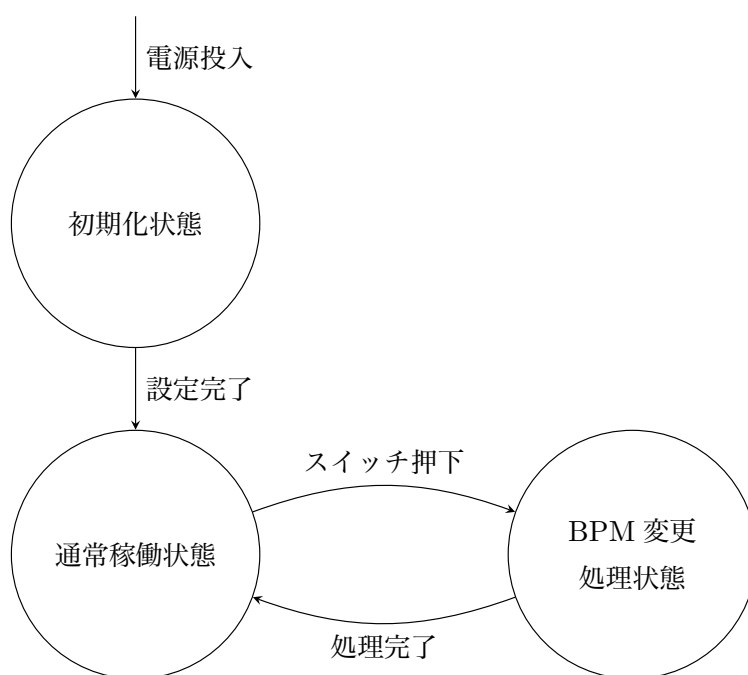


図 3.2 同期サーバーの状態遷移図

第 4 章

詳細設計：同期

4.1 部品一覧

本システムの構築に使用するハードウェアおよびソフトウェアの構成を表 4.1 に示す。各部品は、リズム管理の正確性と Wi-Fi 通信の安定性を考慮して選定している。

表 4.1 同期サーバーの構成部品一覧

区分	名称・型番	役割
メインボード	Arduino UNO R4 WiFi	システムの主制御、および Wi-Fi 通信、LED マトリクス表示を担当します。
入力部品	スイッチ	BPM のアップおよびダウンの入力に使用します。
電源・通信機	MacBook Air	USB Type-C ケーブルを介して Arduino へ電力を供給し、シリアル出力を監視します。
ライブラリ	WiFiS3	Arduino UNO R4 WiFi での無線通信を制御するために使用します。
ライブラリ	Arduino_LED_Matrix	内蔵 LED マトリクスへの数値表示を制御するために使用します。

4.2 ハードウェア設計

本システムの全体回路構成を図 4.1 に示す。

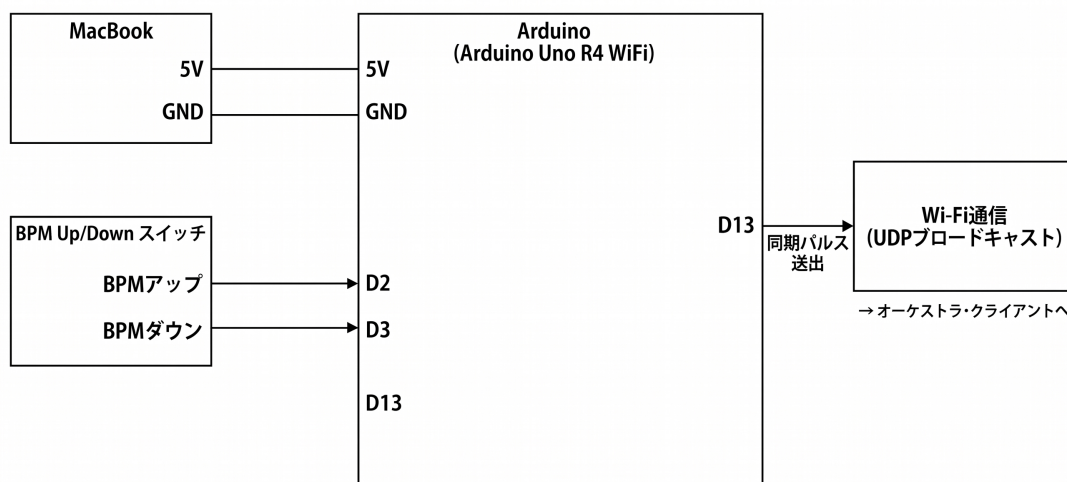


図 4.1 同期システムの全体構成図

図 4.1 は、Arduino UNO R4 WiFi を中心とした最小限の構成を表す。BPM の操作入力にはデジタルピンの D2 および D3 を使用し、Arduino 内部のプルアップ抵抗 ($INPUT_PULLUP$) を有効にすることで、外付け抵抗を省略し回路の簡略化と信頼性の向上を図っている。

電源は MacBook より USB Type-C ケーブルを経由して供給される。スイッチが押下されると、該当するピンの状態が LOW になることをソフトウェア側で検知し、BPM の増減処理を行う。

LED マトリクスはマイコンボードに内蔵されているため、外部配線を必要とせずに現在の設定値を視覚化することが可能である。

4.3 ソフトウェア設計とファイル構成

本システムは単一のスケッチファイルである「server.ino」で構成される。サーバー側で実装する主要な関数の仕様を表 4.2 に示す。表 4.2 に定義された各関数は、演奏のリアルタイム性を維持するために、処理時間を最小限に抑えるよう設計する。

表 4.2 サーバー側の主要関数一覧

関数名	役割
setup()	通信ポートの開放, LED マトリクスの初期化, および内部変数の初期設定を行います。
Bar_control()	現在時刻をミリ秒単位で監視し, BPM に応じた間隔で小節番号をブロードキャスト送信します。
BPM_control()	D2 および D3 ピンの状態を監視し, BPM の増減, LED 表示の更新, および変更通知の送信を行います。

4.4 データ設計

システム内部で保持する主要な変数の定義を表 4.3 に示す。小節番号および BPM 値は、通信負荷を軽減するため 1 バイトのバイナリデータとして扱う。表 4.3 に示す通り、BPM の下限を 40 に設定し、小節番号の上限を 39 とすることで、受信側（クライアント）においてデータ種別の判定を容易にする。

表 4.3 主要変数の定義

変数名	型	範囲	内容
bpm	uint8_t	40 ～ 255	現在の演奏テンポを格納します。
bar_count	uint8_t	0 ～ 39	現在の小節番号を格納します。
interval	uint32_t	算出値	BPM に基づき, 1 小節のミリ秒時間を保持します。

4.5 処理フロー

本システムのメインループおよび各関数の処理手順について説明する。

4.5.1 全体処理フロー

本システムのメインプログラムにおける全体の処理フローを図 4.2 に示す。図 4.2 は、電源投入後の初期化から、メインループ内での関数呼び出しの流れを表したものである。setup 関数での初期化完了後、loop 関数内では小節管理を行う Bar_control 関数と、BPM 制御を行う BPM_control 関数を継続的に呼び出し、リアルタイムな同期処理を実現する。

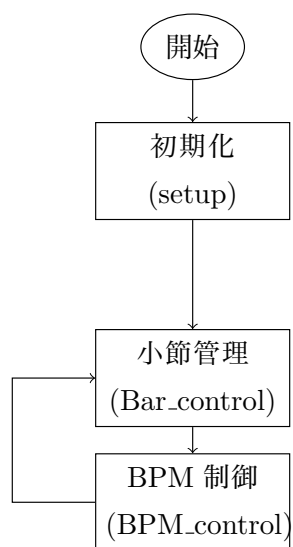


図 4.2 全体処理フローチャート

4.5.2 小節管理処理 (Bar_control)

小節番号のカウントおよび送信を担う Bar_control 関数の処理フローを図 4.3 に示す。図 4.3 は、時間経過に基づいた同期信号の送出ロジックを示している。現在の時刻をミリ秒単位で取得し、前回の送信時刻からの経過時間が現在の BPM に基づく「1 小節の時間間隔」に達したかを判定する。条件を満たした場合、小節番号を更新し、UDP ブロードキャストを用いて全クライアントへ送信を行う。

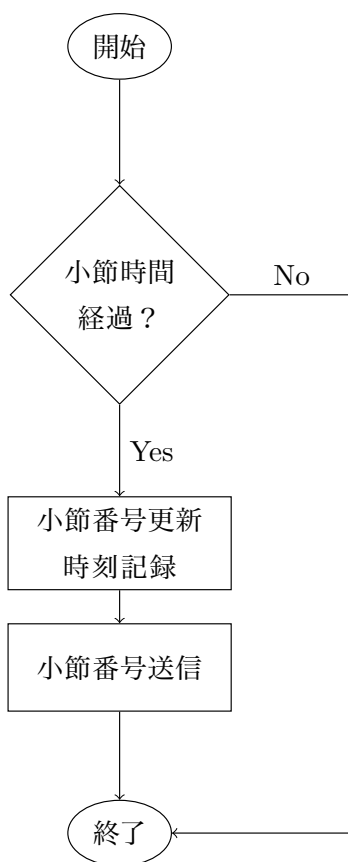


図 4.3 Bar_control 関数のフローチャート

4.5.3 BPM 制御処理 (BPM_control)

ユーザー入力进行处理する BPM_control 関数の処理フローを図 4.4 に示す。図 4.4 は、物理スイッチの入力検知から BPM 更新までの流れを示している。デジタルピン D2 または D3 の入力状態を監視し、スイッチの押下を検知した際に BPM 値を 5 ずつ増減させる。更新された BPM 値は即座に内蔵 LED マトリクスへ表示される。また、ネットワークの負荷を軽減するため、前回の送信から 1 秒以上経過している場合のみ、新しい BPM 値をクライアントへ通知する仕組みとして

いる.

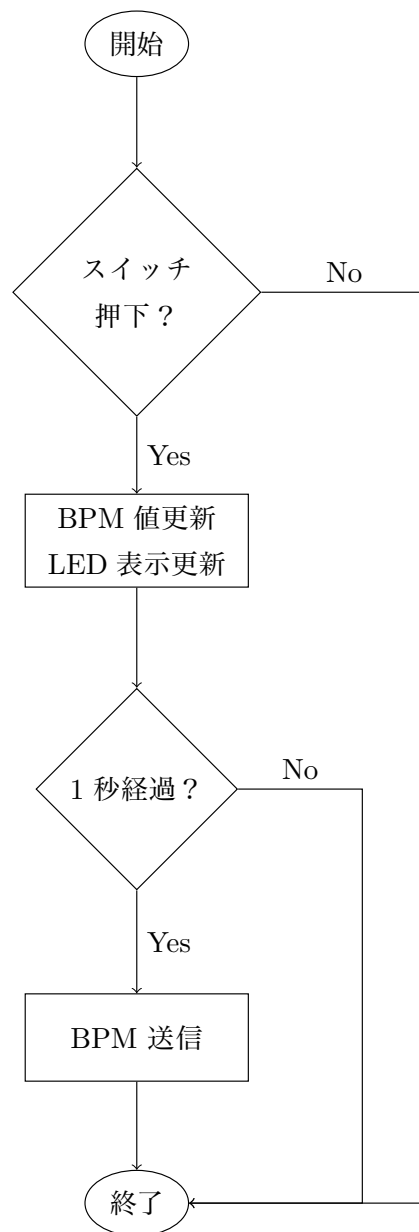


図 4.4 BPM_control 関数のフローチャート